```
// ====================================================================
// STRIPE PROJECT
// Annexe 1.B : OLTP Physical Data Model
// Version : v36
// Format : DBML (Database Markup Language) - https://dbml.dbdiagram.io
// Cas d'usage métier : Stripe Business Case / OLTP
// Objectif : schéma normalisé (3NF), intégrité ACID, débit élevé, détection de fraude en temps réel.
// Nomenclature : noms de tables en PascalCase, colonnes en snake_case.
// Compatible : dbdiagram.io, dbml-cli (dbml2sql), DBML VS Code.
// v26 : ajout de l'état "suspended" dans TransactionStatus.
// v26 : scission du groupe "Security & Compliance" en trois parties:
//       - Sécurité (#922B21) : ChangeAuditLog, SecurityIncidentStatus, SecurityIncident ;
//       - Conformité (#B7950B) : ConsentRecord, Regulation, DataSubjectRequestStatus, DataSubjectRequest.
//       - Audit (#8E44AD) :  AuditActorType, AuditAction, DataAccessLog (infrastructure d'audit partagée).
// v27 : ajout de updated_by/updater_type sur Customer/Merchant/Transaction :
//       ChangeAuditLog ne pouvait pas être dérivée des topics cdc.customer/merchant/transaction
//       tant que ces topics, qui sont eux-mêmes le reflet des tables OLTP, ne portaient pas information.
// v28 : ChangeAuditLog passe du groupe "sécurité" au groupe "audit" ;
// v29 : - nouvelles table "FraudScoringLog" (groupe Transaction, #E74C3C) pour tracer l'historique
//         des prédictions de frude du service de scoring de fraude en temps réel : horodatage, transaction,
//         version et role du modèle, vecteur de features, score, niveau de risque.
// v30 : - nouvelles tables FraudReview et FraudDecision (groupe Transaction, #E74C3C) ;
//       - justification de FraudScore, FraudReview et FraudDecision ;
//       - mutabilité de FraudReview et FraudDecision ;
//       - ChangeAuditLog : révision de la note pour réintégrer les données immuables (FraudScore,
FraudScoringLog) dans le CAL ;
//       - AuditActorType et AuditAction rejoignent le groupe References #95A5A6) ;
//       - correction de updated_by/updater_type (identification polymorphe, non nulle, +  restriction aux types
définis par AuditActorType) ;
//       - correction de reviewer_id/reviewer_type (cf. updated_by/updater_type) ;
//       - déplacement de AuditActorType/AuditAction dans le groupe 'References' #95A5A6) ;
//       - correction d'une erreur d'indexation su FraudScoringLog (prend en compte le role du modèle dans
l'index de clé primaire composée) ;
//       - documentation de l'absence de la table CardNetworkPolicy [levée en v35 : table ajoutée] ;
//       - documentation de l'absence d'attribut précalculé Chargeback.due_at [levée en v34 : due_at ajouté].
// v31 : correctifs de la revue D2/D8.1 (voir CHANGELOG-v31.md) :
//       - identifiants UUID v7 (uuidv7(), PostgreSQL 18) ;
//       - timestamp -> timestamptz (UTC) ;
//       - cohérence des chemins redondants par FK composites (cloisonnement marchand, moyen de paiement du
client,
//         plan tarifaire du marchand, frais fixe de la devise, produit de l'abonnement) ;
//       - [RETIRÉ en v33] partitionnement compatible avec les clés : PK (transaction_id, created_at) ; les
tables filles portent transaction_created_at ; PK composites sur DataAccessLog / ChangeAuditLog / FraudScoringLog
;
//       - identifiants UUID v7 (uuidv7(), PostgreSQL 18) ;
//       - timestamp -> timestamptz (UTC) ;
//       - montants en unités mineures (bigint) ;
//       - updated_by/updater_type étendus à toutes les tables mutables et à ConsentRecord ;
//       - PaymentMethod : token_reference, fingerprint ;
//       - Transaction : nouvelle table IdempotencyKey ;
//       - FraudScore -> FraudScoringLog (FK composite de cohérence) ;
//       - FraudDecision.source_review_id (provenance du label) ;
//       - ChargebackReasonCode.network_code référence CardNetwork ;
//       - AuditActorType.is_human nullable ;
//       - index : retrait de Transaction(created_at) et Transaction(status_code) ; index partiels dans le DDL
complémentaire ;
//       - CHECK, EXCLUDE, index partiels, immutabilité, rôles, CDC : voir 02_complement.sql.
// v32 : correctif :
//       - PaymentMethod.customer_id : référence simple vers Customer, supprimée à tort en v31.
// v33 : RETRAIT DU PARTITIONNEMENT (décision de conception, voir CHANGELOG-v33.md) :
//       - PK simples (transaction_id ; logentry_id) ; les tables filles ne portent plus transaction_created_at ;
//       - FK simples vers Transaction(transaction_id) rétablies en ligne ;
//       - index temporels created_at / evaluated_at / occurred_at rétablis (balayages temporels et purge de
rétention) ;
//       - FraudScore -> FraudScoringLog : la FK composite porte désormais transaction_id (en v31/v32 elle ne
//         garantissait pas que l'entrée de journal appartienne à la même transaction) ;
//       - rétention des journaux par purge contrôlée (purge_expired_log, 02_complement.sql) et non plus par
suppression de partition ;
```

```
//       - libellé de la section des FK composites clarifié (cohérence des chemins redondants).
// v34 : - Chargeback.due_at ajouté : date limite de réponse précalculée (created_at +
ChargebackReasonCode.response_deadline_days),
//         sur le modèle de DataSubjectRequest.due_at ; contrôle d'écart par la vue v_chargeback_due_discrepancy
(02_complement.sql).
// v35 : - CardBrand renommée CardNetwork ;
//       - brand_code -> network_code (CardNetwork, PaymentMethod) ;
//       - ChargebackReasonCode.card_network -> network_code ;
//       - PaymentMethod.token_ref -> token_reference ;
//       - FraudScoringLog.logentry_id et FraudScore.logentry_id -> fraud_scoring_logentry_id ;
//       - nouvelle table CardNetworkPolicy : délai de réponse aux litiges propre à chaque réseau de cartes ;
//         ChargebackReasonCode.response_deadline_days retiré (le délai dépend du réseau, non du code motif) ;
//         Chargeback.due_at est désormais dérivé de CardNetworkPolicy, via ChargebackReasonCode.network_code ;
// v36 : - Customer.country_code nullable -> not nullable ;
//       - nommage des index.
// ================================================================

Project stripe_oltp {
  database_type: "PostgreSQL"
  Note: '''
    Modèle Transactionnel Stripe.
    - Clés primaires en UUID (génération distribuée, sharding) ;
    - Entité centrale : Transaction ;
    - Fraude, remboursements et rétrofacturations découplés pour ne pas verrouiller la table transactionnelle ;
    - Énumérés à métadonnées gérés par tables de référence (clé naturelle lisible) ;
    - Certains énumérés courts et figés sont laissés en varchar avec contrainte CHECK à l'implémentation ;
    - PricingPlan / PricingPlanFee et Transaction.pricing_plan_id / fee_amount tracent la commission Stripe ;
    - Aucune FK en cascade (NO ACTION partout) : la rétention légale interdit de supprimer des données
financières ;
      l'effacement RGPD passe par pseudonymisation/crypto-shreding ;
    - DBML ne sait pas exprimer certaines contraintes : CHECK, EXCLUDE, index partiels, triggers, privilèges ;
    - Partitionnement : aucune table n'est partitionnée ;
    - Horodatages : tous en timestamptz, stockés en UTC. Le fuseau de session du serveur et des clients doit
aussi être UTC
      (ALTER DATABASE ... SET timezone TO 'UTC') pour que CURRENT_DATE, date_trunc et les conversions de date
soient reproductibles.
      Aucun fuseau métier (marchand, client) n'est porté par le modèle : l'OLAP raisonne en dates UTC (voir D3).
  '''
}

// ================================================================
// RÉFÉRENTIELS (GÉOGRAPHIQUES, MONÉTAIRES, ETC.)
// ================================================================

Table Country [headercolor: #95A5A6] {
  country_code varchar(2) [pk, note: "Code ISO 3166-1 alpha-2"]
  name varchar(100) [not null]
  region varchar(50) [note: "Dénormalisation assumée : regroupement analytique"]

  Note: "Référentiel des pays / régions. Donnée statique."
}

Table Currency [headercolor: #95A5A6] {
  currency_code varchar(3) [pk, note: "Code ISO 4217"]
  name varchar(50) [not null]
  decimals smallint [not null, default: 2, note: "Exposant des unités mineures selon la convetion ISO 4217
(JPY=0, BHD=3)"]

  Note: '''
  Référentiel pur des devises. Donnée statique, sans taux de change.

  decimals : tous les montants sont stockés en bigint = montant x 10^decimals
  '''
}

// ..............................................................
// ACTEURS & ACTIONS AUTITABLES

Table AuditActorType [headercolor: #95A5A6] {
  actor_type_code varchar(20) [pk, note: "user | service | system | merchant_api"]
  label varchar(100) [not null]
```

```
  is_human boolean [note: "Distingue les acteurs humains des acteurs automatiques"]
  is_active boolean [not null, default: true]

  Note: '''
    Nature de l'acteur à l'origine d'un événement audité:
    - user = utilisateur authentifié agissant via une interface (front-end, back-office, etc.) :
      client de la plateforme, agent de support , administrateur interne.
      C'est toujours un humain : is_human = true.
    - service = microservice ou composant applicatif interne agissant de façon autonome ;
      exemple : le service de facturation qui déclenche un prélèvement d'abonnement à échéance, sans intervention
humaine directe à cet instant précis.
      C'est toujours un non-humain : is_human = false.
    - system = un processus système/plateforme au sens large : job planifié, traitement batch, consommateur CDC.
      C'est toujours non-humain : is_human = false.
    - merchant_api = un appel effectué via une interface de marchand par ou pour le compte d'un marchand.
      Pour ce type d'acteur, 'is_human' est variable :
      il peut s'agir d'un appel automatisé (intégration technique du marchand, webhook, synchronisation
programmée)
      ou d'u  appel déclenché manuellement par un employé du marchand via un script ou un client de l'interface.

    Remarque sur la différence entre 'service' et 'system' :
    'service' désigne un composant métier applicatif identifiable (le service de facturation, le moteur de
scoring),
    tandis que 'system' désigne l'infrastructure agissant en tâche de fond, sans logique métier propre.

    Remarque :
    is_human est indéterminé pour merchant_api, sa valeur réelle dépendant de l'acteur lui-même (et non de son
type).
  '''
}

Table AuditAction [headercolor: #95A5A6] {
  action_code varchar(30) [pk, note: "read | create | update | delete | export | login | grant | revoke"]
  label varchar(100) [not null]
  is_sensitive boolean [not null, default: false, note: "Déclenche une surveillance renforcée"]
  requires_justification boolean [not null, default: false, note: "Motif métier obligatoire"]
  is_active boolean [not null, default: true]

  Note: '''
    Types d'actions auditables.

    Actions de lecture (cf. DataAccessLog) :
    - read : consultation d'une ressource (fiche client, détail d'une transaction, etc.) ;
    - export : extraction/téléchargement de données ;
      C'est toujours une operation critique (is_sensitive = true) et justifiable (requires_justification = true)
;
    - login : tentative d'accès au système (qu'elle soit réussie ou échouée) ;
      une action de login n'est pas, en soi, une opération sensible ;
      il faudra l'associer au résultat de l'authentification pour déterminer la nature critique de l'action (cf.
'was_authorized' dans DataAccessLog) ;

    Actions d'écriture (cf. ChangeAuditLog) :
    - create, update, delete : opérations de mutation classiques sur une ligne d'une table auditée ;
      C'est toujours une operation critique (is_sensitive = true) ;
    - grant, revoke : modification d'un état d'autorisation ;
      Ce sont toujours des opérations critiques (is_sensitive = true) et justifiables (requires_justification =
true).

    Les operations critiques sont censées déclencher une alerte en temps réel.
    Les operations justifiables sont obligatoirement accompagnées d'un motif métier (cf. 'justification' dans
DataAccessLog).

    Note v30 : rejoins le groupe "Références" pour suivre AuditActorType.
  '''
}

// ====================================================================
// CLIENTS, PAIEMENTS ET ABONNEMENTS
// ====================================================================

Table Customer [headercolor: #27AE60] {
```

```
  customer_id uuid [pk, default: `uuidv7()`]
  merchant_id uuid [ref: > Merchant.merchant_id, not null]
  email varchar(320) [not null]
  country_code varchar(2) [ref: > Country.country_code, not null]
  created_at timestamptz [not null, default: `now()`]
  updated_by varchar(255) [not null, note: "Identité de l'auteur de la dernière modification"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null]

  Indexes {
    (merchant_id, customer_id)    [unique, name: "uq_customer_merchant_customer", note: "Cible des FK composites
(cloisonnement par marchand)"]
    (merchant_id, `lower(email)`) [unique, name: "uq_customer_merchant_email_ci", note: "v31 : unicité insensible
à la casse"]
  }

  Note: '''
    Modélise la RELATION client-marchand, non la personne physique.
    Une même personne achetant chez trois marchands produit trois
    lignes distinctes : cloisonnement voulu (confidentialité, RGPD,
    responsabilité de traitement).

    updated_by/updater_type : alimente ChangeAuditLog.actor_id/actor_type_code.
  '''
}

Table PaymentMethodType [headercolor: #27AE60] {
  type_code varchar(20) [pk, note: "card | bank_transfer | wallet | direct_debit"]
  label varchar(100) [not null]
  requires_card_details boolean [not null, default: false]
  sort_order smallint
  is_active boolean [not null, default: true]

  Note: "Types de moyens de paiement supportés."
}

Table CardNetwork [headercolor: #27AE60] {
  network_code varchar(20) [pk, note: "visa | mastercard | amex | discover | ..."]
  label varchar(100) [not null]
  is_active boolean [not null, default: true]

  Note: '''
    Réseaux de cartes. Les règles propres à un réseau (délai de réponse aux litiges) sont portées par
CardNetworkPolicy.
  '''
}

Table PaymentMethod [headercolor: #27AE60] {
  payment_method_id uuid [pk, default: `uuidv7()`]
  customer_id uuid [ref: > Customer.customer_id, not null]
  type_code varchar(20) [ref: > PaymentMethodType.type_code, not null]
  network_code varchar(20) [ref: >? CardNetwork.network_code, note: "Réseau de la carte. Null si non applicable"]
  card_last4 char(4) [note: "Jamais le PAN complet (conformité PCI-DSS)"]
  expiry date [note: "Dernier jour du mois d'expiration"]
  token_reference varchar(255) [not null, note: "Jeton débiteur donné en amont par le coffre de tokens ou par
réseau (jamais le PAN)"]
  fingerprint varchar(64) [note: "v31 : empreinte salée non réversible de la carte (dédoublonnage, vélocité
inter-marchands)"]
  is_default boolean [not null, default: false]
  created_at timestamptz [not null, default: `now()`]
  updated_by varchar(255) [not null, note: "Auteur de la création ou de la dernière modification (alimente
ChangeAuditLog.actor_id)"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null]

  Indexes {
    (customer_id, payment_method_id) [unique, name: "uq_pm_customer_pm", note: "Cible des FK composites"]
    (token_reference) [unique, name: "uq_paymentmethod_token_reference"]
    (fingerprint)     [name: "ix_paymentmethod_fingerprint"]
    (customer_id)     [name: ""]
  }

  Note: "Moyens de paiement enregistrés. Aucune donnée de carte sensible en clair."
```

```
}

Table SubscriptionStatus [headercolor: #27AE60] {
  status_code varchar(20) [pk, note: "active | paused | past_due | canceled"]
  label varchar(100) [not null]
  is_billable boolean [not null, default: false, note: "Génère des prélèvements récurrents"]
  is_terminal boolean [not null, default: false, note: "Statut final (canceled)"]
  triggers_dunning boolean [not null, default: false, note: "Déclenche la procédure de relance (past_due)"]
  sort_order smallint
  is_active boolean [not null, default: true]

  Note: '''
    Statuts du cycle de vie d'un abonnement.
    past_due est l'état le plus délicat : le prélèvement a échoué,
    l'abonnement n'est ni actif ni résilié - il déclenche une procédure
    de relance échelonnée (dunning) avant résiliation automatique.
  '''
}

Table Subscription [headercolor: #27AE60] {
  subscription_id uuid [pk, default: `uuidv7()`]
  customer_id uuid [not null, note: "FK composite"]
  product_id uuid [not null, note: "FK composite"]
  merchant_id uuid [not null, note: "Dénormalisation contrôlée (même marchand client/produit). FK composite"]
  payment_method_id uuid [not null, note: "Moyen de paiement autorisé pour le prélèvement récurrent (du même
client). FK composite"]
  status_code varchar(20) [ref: > SubscriptionStatus.status_code, not null, default: "active"]
  billing_cycle varchar(20) [not null, note: "CHECK (billing_cycle IN ('weekly','monthly','yearly')) - liste
courte et figée"]
  amount bigint [not null, note: "Montant prélevé à chaque échéance, en unités mineures de currency_code (CHECK >
0)"]
  currency_code varchar(3) [ref: > Currency.currency_code, not null]
  started_at timestamptz [not null, default: `now()`]
  current_period_end timestamptz [note: "Fin de la période payée en cours"]
  canceled_at timestamptz [note: "Date de demande de résiliation"]
  failed_attempt_count smallint [not null, default: 0, note: "Tentatives de prélèvement échouées consécutives"]
  updated_by varchar(255) [not null, note: "Auteur de la création ou de la dernière modification (alimente
ChangeAuditLog.actor_id)"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null]

  Indexes {
    (customer_id, subscription_id, product_id) [unique, name: "uq_sub_customer_sub_product", note: "Cible de la
FK composite Transaction -> Subscription"]
    (customer_id)        [name: "ix_subscription_customer"]
    (product_id)         [name: "ix_subscription_product"]
    (payment_method_id)  [name: "ix_subscription_payment_method"]
    (status_code)        [name: "ix_subscription_status"]
    (current_period_end) [name: "ix_subscription_period_end", note: "Identifier les échéances à prélever"]
  }

  Note: '''
    Accord de paiement récurrent : le client autorise un marchand à le
    débiter à intervalle régulier jusqu'à résiliation.

    SERVICE DU MARCHAND, INFRASTRUCTURE DE STRIPE : la relation
    d'abonnement (offre, prix, conditions) appartient au marchand ;
    Stripe fournit le mécanisme d'exécution (stockage tokenisé du moyen
    de paiement, déclenchement des prélèvements, gestion des échecs).

    canceled_at marque la DEMANDE de résiliation ; l'abonnement reste généralement actif jusqu'à
current_period_end.

    merchant_id : la contrainte (merchant_id = Customer.merchant_id = Product.merchant_id) est garantie par FK
composites.
    payment_method_id : la contrainte (customer_id = PayementMethod.customer_id) est garantie par FK composite.
  '''
}

// ======================================================================
// MARCHANDS ET PRODUITS
// ======================================================================
```

```
Table MerchantStatus [headercolor: #3498DB] {
  status_code varchar(20) [pk, note: "active | suspended | closed"]
  label varchar(100) [not null]
  can_process_payments boolean [not null, default: false, note: "Encaissement autorisé dans cet état"]
  sort_order smallint
  is_active boolean [not null, default: true]

  Note: "Statuts du cycle de vie d'un marchand."
}

Table Merchant [headercolor: #3498DB] {
  merchant_id uuid [pk, default: `uuidv7()`]
  legal_name varchar(255) [not null]
  country_code varchar(2) [ref: > Country.country_code, not null]
  status_code varchar(20) [ref: > MerchantStatus.status_code, not null, default: "active"]
  created_at timestamptz [not null, default: `now()`]
  updated_by varchar(255) [not null, note: "Identité de l'auteur de la dernière modification"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null]

  Indexes {
    country_code
    status_code
  }

  Note: '''
    Entité qui encaisse les paiements. Rôle réglementé (KYC/KYB,
    responsabilité en cas de litige, compte de reversement) -
    d'où une entité distincte de Customer.

    updated_by/updater_type : voir Customer.
  '''
}

Table Product [headercolor: #3498DB] {
  product_id uuid [pk, default: `uuidv7()`]
  merchant_id uuid [ref: > Merchant.merchant_id, not null]
  name varchar(255) [not null]
  unit_price bigint [not null, note: "Prix en unités mineures de currency_code (CHECK >= 0)"]
  currency_code varchar(3) [ref: > Currency.currency_code, not null]
  active boolean [not null, default: true]
  created_at timestamptz [not null, default: `now()`]
  updated_by varchar(255) [not null, note: "Auteur de la création ou de la dernière modification (alimente
ChangeAuditLog.actor_id)"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null]

  Indexes {
    (merchant_id, product_id) [unique, name: "uq_product_merchant_product", note: "Cible des FK composites"]
    (merchant_id)
  }

  Note: "Catalogue produits par marchand."
}

// ========================================================================
// TARIFICATION ET COMMISSION
// Ajouté suite à la review MCD : sans ces deux tables, la commission
// Stripe (fee_amount_eur côté OLAP) n'avait aucune source traçable
// côté OLTP. Modélise le MCD PRICING_PLAN / BenefitsFrom / IsQuotedIn /
// IsRatedUnder.
// ========================================================================

Table PricingPlan [headercolor: #3498DB] {
  pricing_plan_id uuid [pk, default: `uuidv7()`]
  merchant_id uuid [ref: > Merchant.merchant_id, not null]
  commission_rate decimal(5,4) [not null, note: "Taux de commission, ex. 0.0290 pour 2,90 % - indépendant de
toute devise"]
  effective_from date [not null]
  effective_to date [note: "`null` si le plan est en vigueur (CHECK effective_to > effective_from)"]
  updated_by varchar(255) [not null, note: "Auteur de la création ou de la dernière modification (alimente
ChangeAuditLog.actor_id)"]
```

```
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null]

  Indexes {
    (merchant_id, effective_from)
    (merchant_id, pricing_plan_id) [unique, name: "uq_plan_merchant_plan", note: "Cible de la FK composite
Transaction -> PricingPlan"]
  }

  Note: '''
    Grille tarifaire d'un marchand, versionnée dans le temps (renégociation de commission, changement de
catégorie de risque).

    Un seul plan est en vigueur par marchand à un instant donné.
    Contrainte imposé de façon déclarative par une contrainte d'exclusion PostgreSQL (EXCLUDE USING gist,
extension btree_gist) sur
    (merchant_id, daterange(effective_from, effective_to)) - DBML ne sait pas exprimer cette contrainte.

    commission_rate est indépendant de la devise (un taux reste le même
    quel que soit le montant converti). Le frais fixe, lui, dépend de la
    devise : voir PricingPlanFee.
  '''
}

Table PricingPlanFee [headercolor: #3498DB] {
  pricing_plan_id uuid [ref: > PricingPlan.pricing_plan_id, not null]
  currency_code varchar(3) [ref: > Currency.currency_code, not null]
  fixed_fee bigint [not null, note: "Frais fixe par transaction en unités mineures de la devise, ex. 30 pour 0,30
$ (CHECK >= 0)"]
  updated_by varchar(255) [not null, note: "Auteur de la création ou de la dernière modification (alimente
ChangeAuditLog.actor_id)"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null]

  Indexes {
    (pricing_plan_id, currency_code) [pk]
  }

  Note: '''
    Grille de frais fixes par devise pour un plan tarifaire donné.

    Un plan peut être libellé dans plusieurs devises, chacune avec son propre prélèvement de frais fixe,
    sans conversion de change nécessaire pour l'appliquer à une transaction réglée dans cette devise.
  '''
}


// ======================================================================
// TRANSACTION - entité centrale
// ======================================================================

Table TransactionStatus [headercolor: #E74C3C] {
  status_code varchar(20) [pk, note: "pending | suspended | failed | refunded | successful"]
  label varchar(100) [not null, note: "Libellé affichable"]
  is_terminal boolean [not null, default: false, note: "Statut terminal (plus d'évolution possible)"]
  allows_refund boolean [not null, default: false, note: "Un remboursement est-il autorisé?"]
  sort_order smallint
  is_active boolean [not null, default: true, note: "Retrait de transaction sans suppression"]

  Note: '''
    Etats d'une transaction (statut terminal, autorisation de remboursement , désactivation).
    Porte les logiques de métier qu'une contrainte CHECK ne pourrait pas exprimer.
  '''
}

Table Transaction [headercolor: #E74C3C] {
  transaction_id uuid [pk, default: `uuidv7()`]
  merchant_id uuid [not null, note: "FK composite"]
  customer_id uuid [not null, note: "FK composite"]
  payment_method_id uuid [not null, note: "Moyen de paiement du client. FK composite"]
  product_id uuid [note: "Produit du catalogue marchand acheté par le client - null si paiement libre hors
catalogue. FK composite"]
  subscription_id uuid [note: "Renseigné si prélèvement récurrent (même client, même produit), nul si paiement
```

```
ponctuel. FK composite. CHECK : product_id obligatoire si subscription_id"]
  pricing_plan_id uuid [not null, note: "Plan tarifaire du marchand, avec frais fixe défini pour currency_code.
FK composites"]
  is_merchant_initiated boolean [not null, default: false, note: "Vrai pour un prélèvement d'abonnement : aucun
signal comportemental client exploitable par les modèles de fraude"]
  amount bigint [not null, note: "Montant en unités mineures de currency_code (CHECK > 0)"]
  fee_amount bigint [not null, note: "Commission Stripe en unités mineures = ROUND(amount x commission_rate) +
fixed_fee (arrondi demi-supérieur). Valeur dérivée, matérialisée pour la performance - voir MCD règle #24."]
  currency_code varchar(3) [ref: > Currency.currency_code, not null]
  status_code varchar(20) [ref: > TransactionStatus.status_code, not null]
  ip_geolocation varchar(45) [note: "Adresse IP de l'appelant (IPv4/IPv6, 45 car. max.)"]
  device_type varchar(20) [note: "CHECK (device_type IN ('mobile','desktop','tablet')) - liste courte et figée"]
  created_at timestamptz [not null, default: `now()`]
  updated_by varchar(255) [not null, note: "Auteur de la dernière écriture (création ou changement de statut)"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null]

  Indexes {
    (merchant_id, created_at) [name: "ix_transaction_merchant_created"]
    (customer_id, created_at) [name: "ix_transaction_customer_created"]
    (payment_method_id)       [name: "ix_transaction_payment_method"]
    (product_id)              [name: "ix_transaction_product"]
    (subscription_id)         [name: "ix_transaction_subscription"]
    (created_at)              [name: "ix_transaction_created", note: "Balayages temporels transverses
(rapprochement, reporting, fraude)"]
  }

  Note: '''
    Table centrale du modèle transactionnel.

    Auto-suffisante au sens où montant, devise et date permettent les conversions a posteriori du côté
analytique.

    Clé primaire (PK) = transaction_id seul (pas de partitionnement de la table (décision de conception)) ;
    Le sharding éventuel se ferait par hash(merchant_id).

    Recommandations (hors périmètre) : partitionnement par merchant_id et/ou par created_at (mensuel) ;
réplication synchrone ; CDC vers OLAP/NoSQL.

    merchant_id : contrainte (merchant_id = Customer.merchant_id) garanti par FK composite ; pas de FK simple
vers Merchant.

    payment_method_id : la contrainte (customer_id = PayementMethod.customer_id) est garantie par FK composite.

    pricing_plan_id référence le plan tarifaire dont bénéficie le marchand; conservé explicitement pour figer
l'historique même si le plan du marchand change rétroactivement.

    fee_amount doit être recalculable à partir de PricingPlan/PricingPlanFee pour le couple (pricing_plan_id,
currency_code) ;
    un éventuel écart signale un plan corrigé rétroactivement.

    Index : Transaction est la table la plus écrite du système et chaque index supplémentaire a un coût réel qui
doit être contre-balancé par un gain sur les accès directs courants ;
    On n'a donc pas d'index dédié sur pricing_plan_id car les accès directs du type "transactions facturées sous
ce plan" utilisés en audit/reporting sont rétrospectifs.

    updated_by/updater_type : voir Customer/Merchant. Bien que, par conception, la table soit peu modifiée après
création,
    ces colonnes couvrent les modifications possibles de status_code.
  '''
}

Table IdempotencyKey [headercolor: #E74C3C] {
  merchant_id uuid [ref: > Merchant.merchant_id, not null]
  idempotency_key varchar(255) [not null, note: "Clé fournie par l'appelant (en-tête Idempotency-Key)"]
  transaction_id uuid [ref: > Transaction.transaction_id, not null, note: "FK DEFERRABLE INITIALLY DEFERRED dans
le DDL"]
  created_at timestamptz [not null, default: `now()`]

  Indexes {
    (merchant_id, idempotency_key) [pk]
    (created_at) [name: "ix_idempotencykey_created", note: "Purge opérationnelle (fenêtre de rejeu, ex. 24 h à 30
```

```
j)"]
  }

  Note: '''
    Garantit qu'un rejeu de la même requête de paiement ne crée pas deux transactions.
    Table séparée : donnée opérationnelle à durée de vie courte (fenêtre de rejeu), purgeable sans toucher à
Transaction.
    La ligne est insérée dans la même transaction SQL que la Transaction (FK DEFERRABLE INITIALLY DEFERRED).
    Purgeable : opérationnelle, hors périmètre d'audit.
  '''
}

// ======================================================================
// ENTITÉS DÉPENDANTES DE LA TRANSACTION (relations 1:0..1)
// Découplées pour éviter d'alourdir et de verrouiller Transaction.
// ======================================================================

Table Refund [headercolor: #E74C3C] {
  refund_id uuid [pk, default: `uuidv7()`]
  transaction_id uuid [ref: - Transaction.transaction_id, not null, unique]
  amount bigint [not null, note: "Unités mineures de la devise de la transaction (CHECK > 0 & <=
Transaction.amount)"]
  reason varchar(255)
  created_at timestamptz [not null, default: `now()`]
  updated_by varchar(255) [not null, note: "Auteur de la création ou de la dernière modification (alimente
ChangeAuditLog.actor_id)"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null]

  Note: '''
    Remboursement volontaire (au plus un par transaction).
    Table séparée : la plupart des transactions n'en ont pas -
    éviter des colonnes NULL sur une table à fort volume.
  '''
}

//.....................................................................

Table ChargebackStatus [headercolor: #E74C3C] {
  status_code varchar(20) [pk, note: "open | won | lost"]
  label varchar(100) [not null]
  is_terminal boolean [not null, default: false]
  sort_order smallint
  is_active boolean [not null, default: true]

  Note: "Statuts du cycle de contestation, imposé par les réseaux de cartes."
}

Table CardNetworkPolicy [headercolor: #E74C3C] {
  network_code varchar(20) [pk, ref: - CardNetwork.network_code, note: "Au plus une politique par réseau"]
  response_deadline_days smallint [not null, note: "Délai (en jours) imposé au marchand par le réseau pour
répondre à un litige (CHECK > 0). Ce n'est pas le délai RGPD (voir Regulation.response_deadline_days)"]

  Note: '''
    v35. Règles propres à un réseau de cartes. Pour l'instant : le délai de réponse à un litige.
    Ce délai dépend du réseau et non du code motif : il est porté ici plutôt que sur ChargebackReasonCode (3NF).
    Sert à dériver Chargeback.due_at = Chargeback.created_at + response_deadline_days (via
ChargebackReasonCode.network_code).
    Un réseau sans ligne ici n'a pas de délai défini : due_at reste NULL.
    Table de référence, valeurs à alimenter depuis la documentation des réseaux.
  '''
}

Table ChargebackReasonCode [headercolor: #E74C3C] {
  reason_code varchar(20) [pk, note: "Code normalisé par le réseau de cartes"]
  network_code varchar(20) [ref: > CardNetwork.network_code, not null, note: "Réseau de cartes qui définit ce
code (visa, mastercard, amex, etc.)"]
  label varchar(255) [not null]
  category varchar(50) [note: "fraud | authorization | processing_error | consumer_dispute"]
  is_active boolean [not null, default: true]

  Note: '''
```

```
    Les réseaux définissent des dizaines de codes, avec des libellés propres, et la liste évolue.
    Cas typique justifiant une table de référence plutôt qu'un CHECK.
    Pas d'index secondaire : la table reste trop petite (quelques dizaines de lignes) pour qu'un index apporte un
bénéfice sur un balayage séquentiel.

    Le délai de réponse dépend du réseau de cartes mais pas du motif de retro-facturation proprement dit :
    il est porté par CardNetworkPolicy (response_deadline_days), joint par network_code.
  '''
}

//.....................................................................

Table Chargeback [headercolor: #E74C3C] {
  chargeback_id uuid [pk, default: `uuidv7()`]
  transaction_id uuid [ref: - Transaction.transaction_id, not null, unique]
  reason_code varchar(20) [ref: > ChargebackReasonCode.reason_code, not null]
  status_code varchar(20) [ref: > ChargebackStatus.status_code, not null, default: "open"]
  created_at timestamptz [not null, default: `now()`]
  due_at timestamptz [note: "Date limite de réponse, précalculée : due_at = created_at +
CardNetworkPolicy.response_deadline_days du réseau du motif (CHECK : > created_at). NULL si le réseau n'a pas de
politique définie. Valeur dérivée, matérialisée pour le suivi des échéances"]
  resolved_at timestamptz
  updated_by varchar(255) [not null, note: "Auteur de la création ou de la dernière modification (alimente
ChangeAuditLog.actor_id)"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null]

  Indexes {
    status_code [name: "ix_chargeback_status"]
    reason_code [name: "ix_chargeback_reason"]
  }

  Note: '''
    Litige initié par le client auprès de sa banque.
    Cycle de vie propre et mises à jour tardives : isolé pour ne pas verrouiller Transaction sur le chemin
critique de paiement.

    due_at : la date limite de réponse est précalculée comme pour DataSubjectRequest.due_at :
              Chargeback.due_at = Chargeback.created_at + CardNetworkPolicy.response_deadline_days (réseau du
code motif : ChargebackReasonCode.network_code).

    due_at reste NULL si le réseau du code du motif n'a pas de politique définie.
  '''
}

// ======================================================================

Table FraudScoringLog [headercolor: #E74C3C] {
  fraud_scoring_logentry_id uuid [pk, default: `uuidv7()`]
  transaction_id uuid [ref: > Transaction.transaction_id, not null, note: "Admet plusieurs entrées possibles par
transaction (supporte les architectures champion/challenegers)."]
  model_version varchar(20) [not null, note: "Traçabilité et reproductibilité de la prédiction"]
  model_role varchar(20) [not null, note: "Distingue l'entrée qui a piloté la décision (champion) des entrées
d'évaluation parallèle (challengers) - CHECK (model_role IN ('production','staging'))"]
  features jsonb [not null, note: "Vecteur de features tel que lu par le service au moment de la prédiction
(entrée exacte du modèle)"]
  anomaly_score decimal(5,4) [not null, note: "Score produit par le modèle à cet instant, avant toute décision"]
  risk_level_code varchar(10) [ref: > FraudRiskLevel.risk_level_code, not null, note: "Niveau de risque dérivé du
score au moment de la prédiction"]
  evaluated_at timestamptz [not null, default: `now()`, note: "Horodatage de la prédiction - distinct de
Transaction.created_at"]

  Indexes {
    (transaction_id, model_version, model_role) [unique, name: "uq_fsl_txn_model", note: "Un scoring, au plus,
par transaction et par modèle"]
    (fraud_scoring_logentry_id, transaction_id, model_version, anomaly_score, risk_level_code) [unique, name:
"uq_fsl_for_fraudscore", note: "Cible de la FK composite FraudScore -> FraudScoringLog (même transaction, mêmes
score, niveau et version)"]
    (transaction_id, evaluated_at) [name: "ix_fsl_txn_evaluated", note: "Retrouver l'historique des prédictions
d'une transaction donnée (usage conformité)"]
    (evaluated_at) [name: "ix_fsl_evaluated", note: "Balayages temporels"]
  }
```

```
  Note: '''
    Journal des prédictions de fraude par le service de scoring en temps réel.

    Trace chaque prédiction produite par chaque moteur de scoring de fraude en temps réel.
    Une transaction pointe potentiellement sur plusieurs entrées de journal distinctes (transaction_id n'est pas
unique),
    car plusieurs évaluations de la même transaction peuvent être effectuées en même temps par différents moteurs
(en parallèle).

    Typiquement, c'est le cas lors de l'évaluation d'un challenger en mode *shadow*.

    Le rôle d'un modèle ('model_role') distingue l'entrée qui a réellement piloté la décision pour une même
transaction (ex: 'production') des entrées d'évaluations parallèles (ex: 'staging' ou 'shadow').
    Le rôle d'un modèle peut changer dans le temps (ex: promotion 'staging' > 'production').

    (transaction, model_version, model_role) forme une clé d'unicité.

    Rétention alignée sur celle des transactions (FraudScore référence ce journal) : pas de purge autonome.

    IMMUABLE (append-only) : absence de droits UPDATE/DELETE - y compris pour les administrateurs (comme
DataAccessLog/ChangeAuditLog).

    Le jounl est alimenté asynchroniquement par le service de scoring (hors chemin critique de paiement). Ne doit
pas verrouiller Transaction. Ne doit pas dupliquer le coût des écritures dans Transaction.

    Le format jsonb absorbe le vecteur de features tel qu'utilisé par le modèle à l'instant t, sans imposer un
schéma figé par version de modèle - la structure du vecteur de features peut évoluer sans migration de schéma.
  '''
}

//.....................................................................

Table FraudRiskLevel [headercolor: #E74C3C] {
  risk_level_code varchar(10) [pk, note: "low | medium | high"]
  label varchar(50) [not null]
  requires_manual_review boolean [not null, default: false, note: "Déclenche une revue manuelle avant
confirmation"]
  blocks_transaction boolean [not null, default: false, note: "Blocage automatique avant confirmation"]
  sort_order smallint
  is_active boolean [not null, default: true]

  Note: '''
    Porte la logique de décision associée au score de fraude -
    déclenchement d'une revue manuelle ou d'un blocage automatique -
    qu'une contrainte CHECK ne pourrait pas exprimer. Même principe que
    TransactionStatus.allows_refund : la valeur seule ne suffit pas,
    c'est l'action qu'elle déclenche qui justifie la table.

    Locale à cette section plutôt que dans le groupe d'énumérés du haut :
    seule FraudScore la référence, même principe que AuditActorType/
    AuditAction restées locales à la section conformité.
  '''
}

Table FraudScore [headercolor: #E74C3C] {
  fraud_score_id uuid [pk, default: `uuidv7()`]
  transaction_id uuid [ref: - Transaction.transaction_id, not null, unique, note: "Ne retient qu'une seule
évaluation par transaction (quelque soit le nombre de tentatives de scoring produites)"]
  fraud_scoring_logentry_id uuid [not null, note: "Entrée de FraudScoringLog retenue. FK composite"]
  anomaly_score decimal(5,4) [not null, note: "Score 0.0000 à 1.0000"]
  risk_level_code varchar(10) [ref: > FraudRiskLevel.risk_level_code, not null]
  model_version varchar(20) [not null, note: "Traçabilité du modèle ayant produit le score (not null, comme dans
le journal)"]
  evaluated_at timestamptz [not null, default: `now()`]

  Indexes {
    (risk_level_code, evaluated_at) [name: "ix_fraudscore_risk_level", note: "File de revue manuelle : niveau de
risque puis récence"]
  }
```

```
  Note: '''
    Capture le score de fraude d'une transaction évalué en temps réel par le service de scoring, en amont de
l'exécution de la transaction.

    FraudScore recoupe partiellement FraudScoringLog :
    - FraudScore conserve l'évaluation retenue pour justifier une revue manuelle ou une décision de blocage ;
    - FraudScoringLog trace toutes les prédictions de score par le service de scoring.

    FraudScore est IMMUABLE (append-only, absence de droit UPDATE/DELETE).
    L'immuabilité répond à l'exigence de traçabilité d'une décision automatisée.

    Tracé dans ChangeAuditLog au même titre que les autres tables ;
    l'acteur à l'origine de l'insertion est le service de scoring lui-même
    (actor_type_code = 'service', actor_id = constante (pas de colonne updated_by sur cette table immuable)).

    fraud_scoring_logentry_id : contrainte sur (transaction, score, niveau et version) par FK composites.
  '''
}

Table FraudReview [headercolor: #E74C3C] {
  review_id uuid [pk, default: `uuidv7()`]
  transaction_id uuid [ref: > Transaction.transaction_id, not null, note: "Plusieurs revues possibles par
transaction."]
  reason varchar(20) [not null, note: "CHECK (reason IN ('uncertain_score', 'random_sampling', ...))"]
  reviewed_at timestamptz [not null, default: `now()`]
  reviewer_id varchar(255) [not null, note: "Identifiant du réviseur - cf. updated_by"]
  reviewer_type varchar(20) [ref: > AuditActorType.actor_type_code, not null, note: "Type du réviseur - cf.
updater_type"]
  outcome varchar(20) [not null, note: "CHECK (outcome IN ('fraud','legitimate','inconclusive')) - n'aboutit pas
systématiquement à une actualisation de FraudDecision"]
  notes text
  updated_by varchar(255) [not null, note: "Auteur de la création ou de la dernière correction"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null, note: "Permet de dériver
ChangeAuditLog depuis le flux CDC"]

  Indexes {
    (review_id, transaction_id)  [unique, name: "uq_fr_review_txn", note: "Cible de
FraudDecision.source_review_id"]
    (transaction_id, reviewed_at) [name: "ix_fraudreview_txn_reviewed", note: "Balayages temporels"]
  }

  Note: '''
    Procès-verbal d'une procédure de revue d'une transaction.

    Trace les revues de transaction dans le but de prouver la conformité du système au droit RGPD à l'explication
d'intervention humaine sur une décision automatisée
    (ici le système de scoring de fraude). Noter qu'il ne s'agit pas de gérer le cycle de vie de la procédure de
revue elle-même mais de tracer le résultat d'une revue déjà effectuée.

    N'est pas limité à cet usage : une revue peut être déclenchée par un échantillonnage aléatoire, un score
incertain, ou toute autre raison métier.

    Il n'est pas exclu que le réviseur (reviewer) soit un service automatisé plutôt qu'un humain.

    L'enregistrement est MUTABLE.

    reviewer_id/reviewer_type identifient le réviseur (obligatoires à la création de l'enregistrement) ; oui, le
réviseur est le créateur du procès-verbal.
    updated_by/updater_type identifient l'acteur qui a créé l'enregistrement ou qui l'a corrigé (répètent
reviewer_id/reviewer_type à la création) ; voir également  Customer/Merchant/Transaction/FraudDecision.

    Il n'y a pas d'association structurelle vers FraudDecision : une revue peut déclencher une actualisation de
FraudDecision ou rester sans suite.
    Toutes les revues ne débouchent donc pas nécessairement sur une décision ferme, et réciproquement : les
décisions ne viennent pas nécessairement toujours des revues.
  '''
}

Table FraudDecision [headercolor: #E74C3C] {
  decision_id uuid [pk, default: `uuidv7()`]
  transaction_id uuid [ref: - Transaction.transaction_id, not null, unique, note: "Une décision par transaction,
```

```
au plus."]
  source_review_id uuid [note: "Revue à l'origine de la décision, éventuellement nulle (décision par défaut,
décision automatique, etc.). FK composite"]
  decision_value varchar(20) [not null, note: "CHECK (decision_value IN ('legitimate','fraud'))"]
  decision_status varchar(20) [not null, default: 'defaulted', note: "CHECK (decision_status IN
('defaulted','confirmed'))"]
  defaulted_at timestamptz [note: "Horodatage de la labellisation par défaut, faute de revue"]
  confirmed_at timestamptz [note: "Horodatage de la confirmation/infirmation par revue"]
  updated_by varchar(255) [not null, note: "Traçabilité CDC-based via ChangeAuditLog"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null, note: "Distingue actualisation
automatique vs. issue d'une revue"]

  Indexes {
    source_review_id [name: "uq_frauddecision_source_review", note: "Retrouver une décision par sa revue
d'origine"]
  }

  Note: '''
    Enregistrement d'une décision de légitimité ou de fraudulence d'une transaction a posteriori.
    Répond au problème de l'étiquetage différé, en distinguant une décision automatique immédiate
    (autorisation/blocage en cours de paiement) d'une confirmation éventuelle tardive.

    L'absence de décision sur une transaction est traduite par l'absence d'enregistrement.

    MUTABLE avec cycle de vie à deux états :
    - 'defaulted' à la création (décision automatique en cours de paiement),
    - puis, éventuellement, 'confirmed', généralement à la suite d'une revue.

    updated_by/updater_type distinguent une actualisation automatique (service de scoring) d'une actualisation
issue d'une revue humaine ;
    voir également Customer/Merchant/Transaction/FraudReview.
  '''
}

// ======================================================================
// SECURITE ET CONFORMITÉ
// Tables en ÉCRITURE SEULE (append-only) : aucun droit UPDATE ni DELETE,
// même pour les administrateurs. Un journal modifiable ne prouve rien.
// Sources du reporting de conformité (RGPD, PCI-DSS, CCPA).
// ======================================================================

// ======================================================================
// JOURNALISATION DES LECTURES ET DES ECRITURES

Table DataAccessLog [headercolor: #8E44AD] {
  logentry_id uuid [pk, default: `uuidv7()`]
  actor_id varchar(255) [not null, note: "Identité de l'utilisateur ou du service appelant"]
  actor_type_code varchar(20) [ref: > AuditActorType.actor_type_code, not null]
  action_code varchar(30) [ref: > AuditAction.action_code, not null]
  resource_type varchar(50) [not null, note: "Entité consultée : Customer | Transaction | PaymentMethod ..."]
  resource_id varchar(64) [note: "Optionnel : identifiant de l'enregistrement consulté"]
  merchant_id uuid [ref: >? Merchant.merchant_id, note: "Optionnel : l'accès à une ressource peut être
transversal (ex: support Stripe)"]
  customer_id uuid [ref: >? Customer.customer_id, note: "Optionnel"]
  source_ip varchar(45) [note: "Adresse d'origine de la requête appelante (IPv4/IPv6)"]
  user_agent varchar(255) [note: "user-agent de la requête appelante"]
  justification varchar(500) [note: "Motif métier (obligatoire si l'action requiert une justification)"]
  was_authorized boolean [not null, default: true, note: "Faux si la tentative d'accès est refusée (échec
d'authentification ou d'autorisation)"]
  occurred_at timestamptz [not null, default: `now()`]

  Indexes {
    (customer_id, occurred_at) [name: "ix_dal_customer_occurred", note: "Répondre à : qui a consulté les données
de ce client ?"]
    (merchant_id, occurred_at) [name: "ix_dal_merchant_occurred"]
    (actor_id, occurred_at) [name: "ix_dal_actor_occurred"]
    (was_authorized) [name: "ix_dal_was_authorized"]
    (occurred_at) [name: "ix_dal_occurred", note: "Balayages temporels et purge de rétention"]
  }

  Note: '''
```

```
    Historique des accès aux données : qui accéde à quoi, quand, d'où, et pourquoi.

    ACCES EN ECRITURE SEULE (append-only) : aucun droit UPDATE ni DELETE, même pour les administrateurs.

    Volume élevé : purge de rétention par le rôle stripe_retention (fonction purge_expired_log,
02_complement.sql) et archivage à froid au-delà de la durée de conservation.

    Alimenté asynchroniquement pour ne pas pénaliser le chemin critique.

    index ix_dal_was_authorized : l'index partiel recommandé (mais non exprimable en DBML) porte sur le prédicat
(WHERE was_authorized = false) pour cibler les tentatives refusées, plus rares et plus utiles à l'investigation
que la masse des accès légitimes.
    Pas d'index dédié sur 'resource_type' : c'est le plus faible des index candidats (rapport large, moins
fréquent que les besoins par client/marchand/acteur).
  '''
}

// ============================================================================

Table ChangeAuditLog [headercolor: #8E44AD] {
  logentry_id uuid [pk, default: `uuidv7()`]
  table_name varchar(50) [not null, note: "Nom de la table modifiée"]
  record_id varchar(64) [not null, note: "Identifiant de l'enregistrement modifié (dépend du type de table)"]
  action_code varchar(30) [ref: > AuditAction.action_code, not null]
  actor_id varchar(255) [not null]
  actor_type_code varchar(20) [ref: > AuditActorType.actor_type_code, not null]
  changed_fields jsonb [note: "Champs modifiés : { champ: { avant, apres } }"]
  merchant_id uuid [ref: >? Merchant.merchant_id]
  occurred_at timestamptz [not null, default: `now()`]

  Indexes {
    (table_name, record_id, occurred_at) [name: "ix_cal_table_record_occurred"]
    (merchant_id, occurred_at) [name: "ix_cal_merchant_occurred"]
    (occurred_at) [name: "ix_cal_occurred", note: "Balayages temporels et purge de rétention"]
  }

  Note: '''
    Historique des créations, modificiations et suppressions des données transactionnelles.

    Trace également les créations de données immuables (FraudScore, FraudScoringLog, ConsentRecord)
    pour maintenir un chemin d'alimentation unique vers les tables analytiques et les rapports de conformité.

    FraudReview est mutable (tracée comme toute table mutable) ;
    DataAccessLog est elle-même un journal et n'est pas re-journalisée ;
    ChangeAuditLog est exclue de la publication CDC pour éviter une boucle d'écriture.

    ACCES EN ECRITURE SEULE (append-only) : absence de droit UPDATE ou DELETE (y compris pour les
administrateurs).

    Alimenté asynchroniquement par un flux CDC (Change Data Capture) (ex: lecture asynchrone du WAL sous
PostgreSQL).
    En conséquence de quoi, l'insertion d'une entrée de journal reste en dehors du chemin transactionnel.

    Le format jsonb absorbe des structures hétérogènes selon la table auditée, sans imposer un schéma par entité.
  '''
}

// ============================================================================
// CONFORMITE

Table ConsentRecord [headercolor: #B7950B] {
  consent_id uuid [pk, default: `uuidv7()`]
  customer_id uuid [not null]
  merchant_id uuid [not null, note: "Le consentement est donné à un marchand précis (= Customer.merchant_id,
garanti par FK composite v31)"]
  purpose varchar(100) [not null, note: "Finalité : marketing | analytics | data_sharing | profiling"]
  is_granted boolean [not null, note: "Accordé ou retiré"]
  legal_basis varchar(50) [note: "Base légale RGPD : consent | contract | legitimate_interest"]
  source varchar(50) [note: "Canal de recueil : checkout | email | account_settings"]
  recorded_at timestamptz [not null, default: `now()`]
  updated_by varchar(255) [not null, note: "Auteur de la création ou de la création (table immuable) (alimente
```

```
ChangeAuditLog.actor_id)"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null]

  Indexes {
    (customer_id, purpose, recorded_at) [name: "ix_consent_customer_purpose"]
    (merchant_id, recorded_at)          [name: "ix_consent_merchant"]
  }

  Note: '''
    Historique des consentements - append-only : un retrait crée une
    nouvelle ligne, il n'écrase jamais la précédente. L'état courant se
    lit sur l'enregistrement le plus récent par (customer_id, purpose).

    Le consentement est rattaché au MARCHAND, cohérent avec le
    cloisonnement du modèle : un client consent à un marchand, pas à Stripe.
  '''
}

// ..............................................................

Table Regulation [headercolor: #B7950B] {
  regulation_code varchar(20) [pk, note: "GDPR | CCPA"]
  label varchar(100) [not null]
  response_deadline_days smallint [not null, note: "Délai légal de réponse à une demande d'exercice de droit"]
  is_active boolean [not null, default: true]

  Note: '''
    Réglementations de protection des données couvertes. Porte le délai
    légal de réponse (GDPR : 30 jours ; CCPA : 45 jours), pour que
    DataSubjectRequest.due_at soit dérivable de received_at + ce délai
    plutôt que dépendant d'un calcul applicatif non vérifié par le
    schéma - même principe que ChargebackReasonCode.response_deadline_days.
  '''
}

Table DataSubjectRequestStatus [headercolor: #B7950B] {
  status_code varchar(20) [pk, note: "received | in_progress | fulfilled | rejected"]
  label varchar(100) [not null]
  is_terminal boolean [not null, default: false, note: "Statut final (fulfilled, rejected)"]
  sort_order smallint
  is_active boolean [not null, default: true]

  Note: '''
    Statuts du traitement d'une demande d'exercice de droit. Même famille
    de besoin que TransactionStatus/SubscriptionStatus/ChargebackStatus
    (statut de workflow avec état terminal) - traité en table pour rester
    cohérent avec ce précédent déjà établi ailleurs dans le modèle.
  '''
}

Table DataSubjectRequest [headercolor: #B7950B] {
  request_id uuid [pk, default: `uuidv7()`]
  customer_id uuid [not null]
  merchant_id uuid [not null, note: "= Customer.merchant_id, garanti par FK composite v31"]
  request_type varchar(30) [not null, note: "access | rectification | erasure | portability | objection"]
  regulation_code varchar(20) [ref: > Regulation.regulation_code, not null]
  status_code varchar(20) [ref: > DataSubjectRequestStatus.status_code, not null, default: "received"]
  received_at timestamptz [not null, default: `now()`]
  due_at timestamptz [not null, note: "Échéance légale - dérivée de received_at +
Regulation.response_deadline_days"]
  fulfilled_at timestamptz
  rejection_reason varchar(500)
  updated_by varchar(255) [not null, note: "Auteur de la création ou de la dernière modification (alimente
ChangeAuditLog.actor_id)"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null]

  Indexes {
    (customer_id, received_at) [name: "ix_dsr_customer_received"]
    (merchant_id, status_code) [name: "ix_dsr_merchant_status"]
    (status_code)              [name: "ix_dsr_status"]
    (due_at)                   [name: "ix_dsr_due", note: "Alerting sur les échéances légales approchantes"]
```

```
  }

  Note: '''
    Demandes d'exercice des droits (accès, effacement, portabilité...).
    Le rapport de conformité doit prouver le respect des DÉLAIS :
    d'où due_at et fulfilled_at, dont l'écart est l'indicateur audité.
  '''
}

// ======================================================================
// SECURITE

Table SecurityIncidentStatus [headercolor: #922B21] {
  status_code varchar(20) [pk, note: "open | investigating | contained | closed"]
  label varchar(100) [not null]
  is_terminal boolean [not null, default: false, note: "Statut final (closed)"]
  sort_order smallint
  is_active boolean [not null, default: true]

  Note: '''
    Statuts du cycle de traitement d'un incident de sécurité. Même
    famille de besoin que TransactionStatus/SubscriptionStatus/
    ChargebackStatus/DataSubjectRequestStatus - traité en table pour
    rester cohérent avec ce précédent déjà établi ailleurs dans le modèle.
  '''
}

Table SecurityIncident [headercolor: #922B21] {
  incident_id uuid [pk, default: `uuidv7()`]
  incident_type varchar(50) [not null, note: "unauthorized_access | data_breach | anomalous_activity |
failed_auth_burst"]
  severity varchar(20) [not null, note: "low | medium | high | critical"]
  merchant_id uuid [ref: >? Merchant.merchant_id, note: "Null si incident transverse"]
  affected_record_count int [note: "Nombre d'enregistrements concernés"]
  description varchar(1000)
  detected_at timestamptz [not null, default: `now()`]
  notified_at timestamptz [note: "Notification à l'autorité - RGPD : 72 h après détection"]
  resolved_at timestamptz
  status_code varchar(20) [ref: > SecurityIncidentStatus.status_code, not null, default: "open"]
  updated_by varchar(255) [not null, note: "Auteur de la création ou de la dernière modification (alimente
ChangeAuditLog.actor_id)"]
  updater_type varchar(20) [ref: > AuditActorType.actor_type_code, not null]

  Indexes {
    (severity, detected_at) [name: "ix_secincident_severity_detected"]
    (status_code)           [name: "ix_secincident_status"]
    (detected_at)           [name: "ix_secincident_detected"]
  }

  Note: '''
    Incidents de sécurité détectés. L'écart detected_at / notified_at
    est un point de contrôle direct du RGPD (délai de 72 heures).
  '''
}

// ======================================================================
// FK COMPOSITES ET COHERENCE DES CHEMINS REDONDANTS (v31)
//
// Les FK composites implémentent :
// - le principe de cloisonnement par marchand (Annexe C §1.C.1.1) ;
// - les règles de cohérence MCD #19, #23, #25, #33 :
//   - #19 = cohérence entre souscription et moyen de paiement,
//   - #23 = cohérence entre un plan facturé et le marchand bénéficiaire,
//   - #25 = cohérence entre `commission_rate` et `fixed_fee,
//   - #33 = cohérence entre client et marchand d'une entrée du journal des accès.
//
// Toutes les FK sont NO ACTION (pas de cascade).
// Les FK composites sont MATCH SIMPLE : elles ne sont vérifiées que si toutes leurs colonnes sont non nulles
(cas des colonnes optionnelles product_id, subscription_id, customer_id).
// ======================================================================
```

```
// ---- Cloisonnement marchand / client / produit / abonnement
Ref fk_subscription_customer: Subscription.(merchant_id, customer_id) > Customer.(merchant_id, customer_id)
Ref fk_subscription_product: Subscription.(merchant_id, product_id) > Product.(merchant_id, product_id)
Ref fk_subscription_payment_method: Subscription.(customer_id, payment_method_id) > PaymentMethod.(customer_id,
payment_method_id)// règle MCD #19

// ---- Transaction
Ref fk_transaction_customer: Transaction.(merchant_id, customer_id) > Customer.(merchant_id, customer_id)
Ref fk_transaction_payment_method: Transaction.(customer_id, payment_method_id) > PaymentMethod.(customer_id,
payment_method_id)
Ref fk_transaction_product: Transaction.(merchant_id, product_id) > Product.(merchant_id, product_id)
Ref fk_transaction_subscription: Transaction.(customer_id, subscription_id, product_id) > Subscription.
(customer_id, subscription_id, product_id)
Ref fk_transaction_pricing_plan: Transaction.(merchant_id, pricing_plan_id) > PricingPlan.(merchant_id,
pricing_plan_id)           // règle MCD #23
Ref fk_transaction_pricing_plan_fee: Transaction.(pricing_plan_id, currency_code) > PricingPlanFee.
(pricing_plan_id, currency_code)// règle MCD #25

// ---- Fraude : cohérence de la transaction entre FraudScore et son entrée de journal, et entre décision et
revue
Ref fk_fraudscore_log: FraudScore.(fraud_scoring_logentry_id, transaction_id, model_version, anomaly_score,
risk_level_code) > FraudScoringLog.(fraud_scoring_logentry_id, transaction_id, model_version, anomaly_score,
risk_level_code)
Ref fk_frauddecision_review: FraudDecision.(source_review_id, transaction_id) > FraudReview.(review_id,
transaction_id)

// ---- Sécurité et conformité (règle #33 et dénormalisation de merchant_id)
Ref fk_dataaccesslog_customer_merchant: DataAccessLog.(merchant_id, customer_id) >? Customer.(merchant_id,
customer_id)
Ref fk_consent_customer_merchant: ConsentRecord.(merchant_id, customer_id) > Customer.(merchant_id, customer_id)
Ref fk_dsr_customer_merchant: DataSubjectRequest.(merchant_id, customer_id) > Customer.(merchant_id, customer_id)
```